



TÉCNICO
LISBOA

ALGORITMOS PARA LÓGICA COMPUTACIONAL

**Relatório do 1º projeto - Nurse Scheduling Problem using
SAT/MaxSAT/PB [PT]**

Autor:

Daniel Ferreira (112291)

daniel.s.ferreira@tecnico.ulisboa.pt

2025/2026 – 1º Semestre, P1

Conteúdo

1	Introdução	2
2	Instruções de execução	2
3	Codificação das restrições	2
3.1	Codificação do número mínimo de enfermeiros em cada turno	3
3.2	Codificação de pelo menos um enfermeiro com qualificações de gerência	3
3.3	Codificação de turnos contínuos	3
3.4	Codificação do número de turnos de um enfermeiro por dia	4
3.5	Codificação de número máximo de turnos total	4
3.6	Codificação do custo ótimo	4
3.7	Otimização	5
4	Algoritmo e configurações	5

1 Introdução

Durante este relatório irei descrever a minha solução MaxSAT para o problema do agendamento dos enfermeiros. O objetivo do problema é construir um horário para um grupo de enfermeiros que têm de cobrir turnos de trabalho que cobrem 24 horas a cada dia. Este planeamento deve considerar os turnos que os enfermeiros preferem e também respeitar as seguintes restrições:

1. Cada turno tem de ter um número mínimo de enfermeiros em serviço $R_{s,d}$ e pelo menos um enfermeiro com qualificações de gerência
2. Cada enfermeiro tem um número mínimo e máximo de turnos que consegue realizar durante cada dia (L_k^{min} e L_k^{max}). Além disso os turnos atribuídos a um enfermeiro em cada um dos dias deverá ser contínuo (sem quaisquer pausas)
3. Cada enfermeiro contém um máximo de turnos que pode trabalhar durante o decorrer do número de dias a serem considerados (H_k)
4. Nenhum enfermeiro deve trabalhar num turno em que declarou que não tinha disponibilidade. Além disso, cada enfermeiro determina o custo de trabalhar em cada um dos turnos em cada dia. O custo do horário final deverá ser minimizado

Nas secções seguintes irei descrever como executar o meu projeto, a forma como codifiquei cada uma das restrições acima e o algoritmo e configurações que usei para obter o melhor resultado possível.

2 Instruções de execução

Para executar a minha solução, basta obter o código do repositório do meu projeto. Verifique que também o módulo PySAT instalado no seu ambiente e com isto, poderá correr o projeto usando `./proj1 < input_file > output_file`.

3 Codificação das restrições

Antes de pensar na codificação de cada restrição, foi importante definir quais variáveis usar para resolver o problema especificado. Para tal decidi criar uma variável $x_{d,s,k}$ que será verdadeira (tomará o valor 1) se o enfermeiro k trabalha no turno número s do dia d . Com estas variáveis será possível verificar que enfermeiros trabalham em cada turno depois de elaborar o horário ótimo.

Nas seguintes subsecções irei descrever como codifiquei cada uma das restrições necessárias para a resolução do problema.

3.1 Codificação do número mínimo de enfermeiros em cada turno

A primeira restrição obriga a que cada turno tenha um número mínimo de enfermeiros a trabalhar. Esse valor mínimo é dado pela variável $R_{s,d}$, em que d é o dia considerado e s o número do turno do dia d .

Codifiquei esta restrição com uma restrição de cardinalidade da seguinte forma:

$$\forall d \in \{1, \dots, D\}, \forall s \in \{1, \dots, S\}, \sum_{k \in N} x_{d,s,k} \geq R_{s,d}$$

em que D e S são o número de dias total e o número de turnos em cada dia e N é o conjunto dos índices dos enfermeiros que têm disponibilidade para trabalhar no turno s no dia d . Não são incluídos todos os enfermeiros como uma simplificação para que sejam incluídas menos variáveis para a codificação desta restrição de cardinalidade, levando à criação de menos cláusulas e variáveis auxiliares (dependendo do algoritmo de codificação utilizado).

3.2 Codificação de pelo menos um enfermeiro com qualificações de gerência

Para obrigar a que exista pelo menos um enfermeiro com qualificações de gerência por turno criei uma cláusula por cada turno que obriga a que pelo menos um enfermeiro qualificado trabalhe nesse turno. Esta restrição pode ser expressa da seguinte forma:

$$\bigwedge_{d=1}^D \bigwedge_{s=1}^S \bigvee_{k \in M} x_{d,s,k}$$

em que D e S são o número de dias total e o número de turnos em cada dia e M é o conjunto dos índices dos enfermeiros que têm disponibilidade para trabalhar no turno s no dia d e que têm qualificações de gerência. A restrição acima codifica uma restrição de cardinalidade com semântica de "pelo menos 1", que é exatamente o que é necessário para codificar esta restrição.

3.3 Codificação de turnos contínuos

Para codificar a restrição de que em todos os dias os turnos atribuídos a um enfermeiro têm de ser consecutivos foi feita a observação de que se o enfermeiro k for trabalhar no turno s do dia d e não for trabalhar no turno $s+1$ do mesmo dia, então não poderá trabalhar em nenhum dos turnos posteriores. Esta condição pode ser expressa da seguinte forma:

$$\bigwedge_{d=1}^D \bigwedge_{k=1}^N \bigwedge_{s_1 \in S} \bigwedge_{\substack{s_2 \in S \\ s_1+1 < s_2}} \neg x_{d,s_1,k} \vee x_{d,s_1+1,k} \vee \neg x_{d,s_2,k}$$

em que D e N são o número de dias total e o número de enfermeiros total e S é o conjunto dos índices dos turnos em que o enfermeiro k tem disponibilidade para trabalhar no dia d . É importante reparar que a expressão $\neg x_{d,s_1,k} \vee x_{d,s_1+1,k} \vee \neg x_{d,s_2,k}$ é equivalente a $(x_{d,s_1,k} \wedge \neg x_{d,s_1+1,k}) \implies \neg x_{d,s_2,k}$

3.4 Codificação do número de turnos de um enfermeiro por dia

Em cada dia, um enfermeiro tem um número mínimo e máximo de turnos que em pode trabalhar definidos pelos valores L_k^{min} e L_k^{max} . Para codificar estas duas restrições foram de novo utilizadas restrições de cardinalidade. Neste caso foram utilizadas duas restrições distintas: uma para codificar que o enfermeiro k deverá trabalhar pelo menos L_k^{min} turnos por dia, e outra para codificar que o enfermeiro k deverá trabalhar no máximo L_k^{max} turnos por dia.

Estas duas restrições podem ser representadas da seguinte forma:

$$\begin{aligned} \forall d \in \{1, \dots, D\}, \forall k \in \{1, \dots, N\}, \quad \sum_{s \in S} x_{d,s,k} &\geq L_k^{min} \\ \forall d \in \{1, \dots, D\}, \forall k \in \{1, \dots, N\}, \quad \sum_{s \in S} x_{d,s,k} &\leq L_k^{max} \end{aligned}$$

em que D e N são o número de dias total e o número de enfermeiros total e S é o conjunto dos índices dos turnos em que o enfermeiro k tem disponibilidade para trabalhar no dia d .

3.5 Codificação de número máximo de turnos total

Além de considerar o número mínimo e máximo de turnos que um enfermeiro pode trabalhar por dia, é também necessário restringir o número máximo de turnos que cada enfermeiro k pode trabalhar ao longo dos D dias, que é dado pelo valor H_k . Para codificar esta restrição foi utilizada mais uma restrição de cardinalidade que se pode representar da seguinte forma:

$$\forall k \in \{1, \dots, N\}, \quad \sum_{d=1}^D \sum_{s \in S} x_{d,s,k} \leq H_k$$

em que D e N são o número de dias total e o número de enfermeiros total e S é o conjunto dos índices dos turnos em que o enfermeiro k tem disponibilidade para trabalhar no dia d .

3.6 Codificação do custo ótimo

Neste problema cada um dos enfermeiros define a prioridade de trabalhar em cada um dos turnos atribuindo-lhes um valor numérico. Se o valor de um dos turnos for -1 então esse enfermeiro mostra que não tem disponibilidade de trabalhar nesse turno. Para codificar esta restrição simplesmente neguei todas as variáveis referentes aos turnos em que cada enfermeiro não tem disponibilidade, ou seja:

$$\bigwedge_{d=1}^D \bigwedge_{s=1}^S \bigwedge_{k \in N} \neg x_{d,s,k}$$

em que D e S são o número de dias total e o número de turnos em cada dia e N é o conjunto dos índices dos enfermeiros que não têm disponibilidade para trabalhar no turno s no dia d .

Para os restantes casos, um enfermeiro k atribui as suas preferências definindo o custo de trabalhar num certo turno s no dia d , dado pelo valor $P_{d,s,k}$. Como o objetivo é minimizar os custos, é preferível colocar enfermeiros a trabalhar em turnos de menor custo. Queremos portanto minimizar o seguinte valor

$$\sum_{d=1}^D \sum_{s=1}^S \sum_{k \in N} P_{d,s,k} * x_{d,s,k}$$

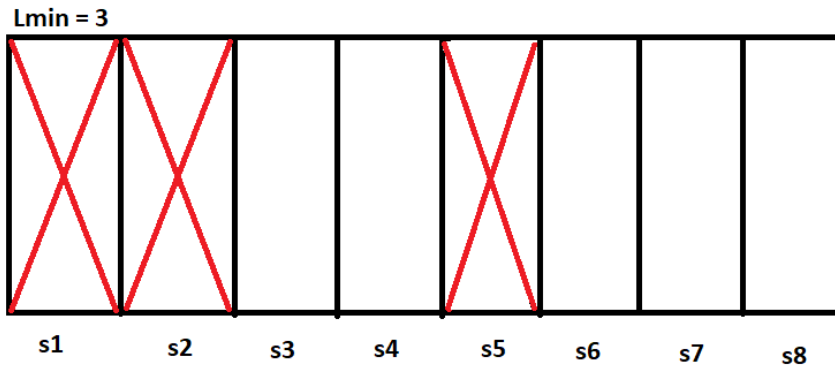
em que N é o conjunto dos índices dos enfermeiros que têm disponibilidade para trabalhar no turno s no dia d .

Como os solvers de MaxSAT tentam sempre minimizar o custo das cláusulas que não são satisfeitas, para cada turno em que o enfermeiro k consegue trabalhar no turno s no dia d é adicionada uma cláusula $\neg x_{d,s,k}$ com peso $P_{k,s,d}$.

3.7 Otimização

Na minha solução, antes de todas as restrições acima serem codificadas, existe um passo de pré-computação que é possível executar devido à seguinte observação: se um enfermeiro k tiver de trabalhar um mínimo de L_k^{min} turnos em cada dia, então, se existir uma sequência isolada de S turnos consecutivos em que o enfermeiro tem disponibilidade para trabalhar em que $S < L_k^{min}$, então o enfermeiro não poderá trabalhar em nenhum dos turnos desse intervalo, pois isso implicaria o enfermeiro k ter de trabalhar em turnos não consecutivos para respeitar o número mínimo de turnos que tem de trabalhar no dia inteiro.

Este exemplo pode ser visualizado na imagem abaixo, onde se descreve com um "X" os turnos em que o enfermeiro não tem disponibilidade durante um dia d . Assumindo que o enfermeiro tem de trabalhar em pelo menos 3 turnos desse dia, isso significa que ele não poderá trabalhar nos turnos 3 e 4, pois fazê-lo implicaria ter de trabalhar num 3º turno que não é consecutivo.



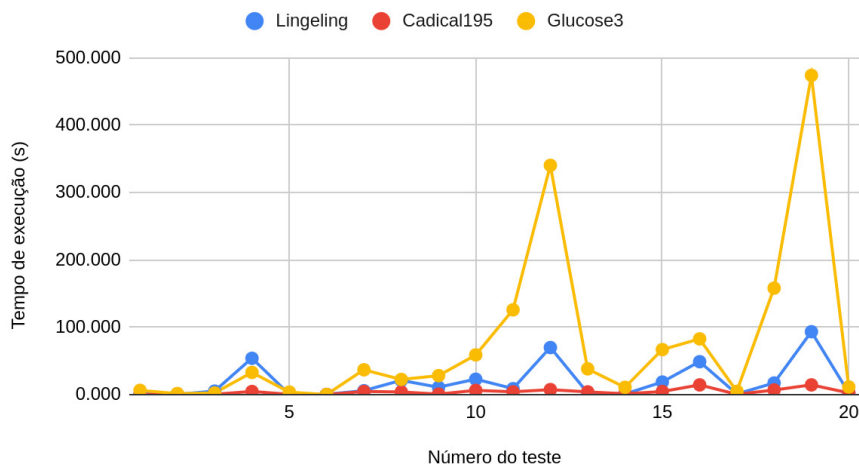
Assim, antes de codificar as restrições do problema, todas as variáveis associadas aos turnos identificados neste passo são negadas, de forma a ter menos variáveis com que trabalhar durante a codificação das restrições.

4 Algoritmo e configurações

Para resolver a fórmula em WCNF que é criada durante a execução do programa é utilizado o MaxSAT solver RC2 do pacote PySAT. Além disso, este solver é inicializado para utilizar o solver cadical195 como o seu oracle. Testei empiricamente os solvers cadical195, gluecose3 e

lingeling e os resultados do tempo de execução encontram-se abaixo, tendo escolhido por fim o solver cadical195 devido à sua performance.

Diferença de tempo de execução entre solvers



Para codificar as restrições de cardinalidade utilizei ou o algoritmo do sequential counter ou o algoritmo totalizer. Para tomar a decisão de qual algoritmo utilizar decidi verificar o número de variáveis que iriam ser utilizadas na codificação da restrição e o valor k a ser utilizado como limite. Caso um destes seja superior a 6 o algoritmo totalizer é utilizado, se não é usado o sequential counter. O valor 6 foi decidido empiricamente.

Esta distinção foi feita pois apesar de teoricamente o número de variáveis auxiliares criadas pelo algoritmo sequential counter ser superior ao totalizer ($O(nk) > O(n\log(k))$) o uso do algoritmo sequential counter mostrou ser mais eficiente para valores baixos de n e k .

Por fim, encontra-se abaixo a diferença entre a solução com ou sem o passo de otimização.

Diferença de execução com e sem otimização

